

DL for ENG

LECTURE 5.2

Sequences and Attention

One question, two answers thirty years apart: how does information get from the start of a long sequence to the end of it?

Remus Teodorescu

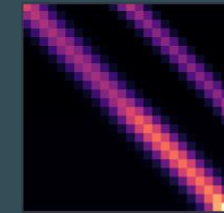
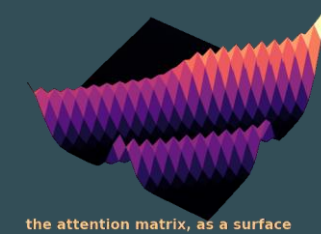
With support from Research Assistant Noman Khan - nomank@energy.aau.dk

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

recurrence · the LSTM · self-attention · the transformer layer

FIVE DAYS OF SUBSTATION DEMAND



the diagonal ridge is recency

the second ridge is the same hour yesterday

seen from above

The series of Ex_05 notebook 04: the next hour, from the last twenty-four.

TWO ROUTES FROM HOUR 1 TO HOUR 24



recurrence — twenty-three steps, through twenty-three activations

Learning Objectives

SECTIONS, IN ORDER

- 1 **Sequences** — what makes a problem sequential, and why the last lecture's layers do not fit
- 2 **Recurrence** — the hidden state, unrolling, and the gradient over time
- 3 **The LSTM** — the cell state, the gates, the equations, sizing
- 4 **Attention** — queries, keys and values; the attention matrix; heads and position
- 5 **Transformers** — the layer, and the three model families
- 6 **Model selection** — the seed spread and the residual check
- 7 **Challenges** — what goes wrong first, and what to check

WHAT WE SKIP, AND WHERE IT IS TAUGHT

training gradients and backpropagation — L6.1

post-training instruction tuning, RL — L6.2

the JetRacer a sequence of frames — L11.1

Reading is split across two books this week. UDL has no recurrent chapter: sections 2 and 3 come from Goodfellow, Bengio & Courville ch. 10; sections 4 and 5 from UDL ch. 12.

Recap

THE TEN QUESTIONS OF L5.1 — CONVOLUTIONAL AND GRAPH NETWORKS

- 1** What kind of network is a convolutional network, and what function does it implement? *UDL ch. 10.1 – 10.2; GBC ch. 9.1*
- 2** Describe the architecture of a convolutional network, layer by layer: what happens to resolution and channels? *UDL ch. 10.3 – 10.4; Ex_05 notebook 01*
- 3** How does a convolutional network differ from a dense network in connections, weights, parameters and symmetry? *GBC ch. 9.2; Ex_05 notebook 01*
- 4** Given those differences, what are convolutional networks used for, and which task needs equivariance, not invariance? *UDL ch. 10.1, 10.5*
- 5** Why does a residual connection help the gradient, and what does a block with nothing to add learn? *UDL ch. 11.2; He et al. (2016)*
- 6** What kind of network is a graph network, and what function does it implement on node features and adjacency? *UDL ch. 13.1 – 13.3*
- 7** Describe the architecture of a graph network: the three steps of a layer, the readout, and K against the diameter? *UDL ch. 13.3, 13.5; Ex_05 notebook 02*
- 8** How does a graph network differ from a dense network, and what does permutation equivariance guarantee? *UDL ch. 13.2; Ex_05 notebook 03*
- 9** Given those differences, what are graph networks used for, and what is the difference between inductive and transductive? *UDL ch. 13.3, 13.6*
- 10** When is a graph network the wrong choice, and what did notebook 03 measure that shows it? *Ex_05 notebook 03*

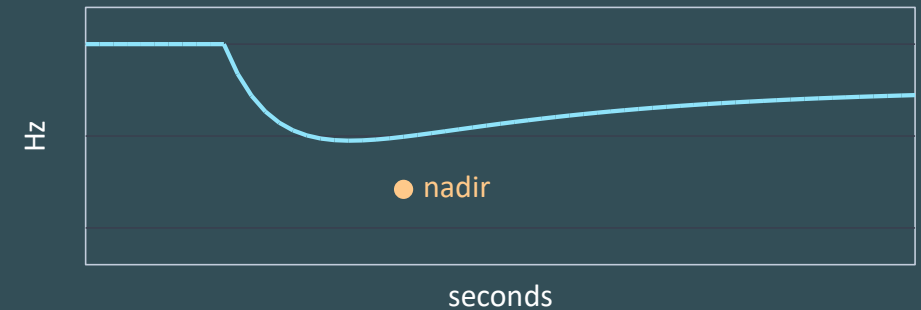
Sequences

ORDER MATTERS, AND THE LENGTH VARIES

- **examples** — a frequency trace, a load profile, a sensor stream
- **order matters** — shuffle the hours and the problem is destroyed
- **the length varies** — laps, runs and outages differ in length
- **convolution needs a fixed neighbourhood** — a sequence has none
- **a dense layer needs a fixed length** — 24 inputs, 24 weights

TALK • 2 MIN Look at the frequency trace on the right. Which earlier seconds does the nadir depend on, and why would padding every event to one length not make it a fixed-size input?

A GRID FREQUENCY EVENT



THE REQUIREMENT

A rule that reads one step at a time and keeps a summary of everything before it. That is a recurrence. The opposite of the six-bus network, where renumbering had to change nothing.

Recurrent Network

A STATE, A RULE FOR UPDATING IT, AND UNROLLING

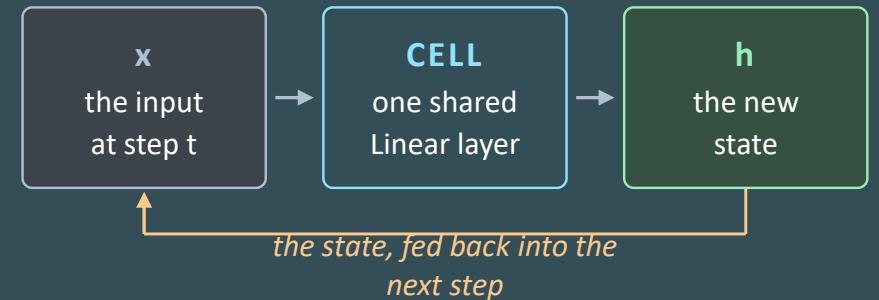
- **the hidden state** — a lossy summary of everything seen so far
- **each step** — combine input and state, apply tanh, update
- **one matrix on the state, one on the input** — the same every step
- **to train, unroll** — one copy of the cell per step, no new parameters
- **weight sharing, a third time** — position, node, and now time step

$$\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b})$$

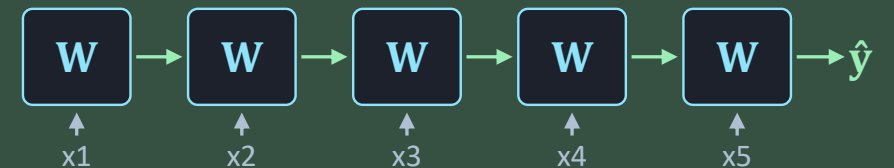
$$\hat{y} = \mathbf{W}_y \mathbf{h}_T + b_y$$

$\mathbf{x}_t$ the input at step t , $\mathbf{h}_t$ the state, $\mathbf{W}_h, \mathbf{W}_x$ shared every step, $\hat{y}$ read from the last state

THE LOOP



UNROLLED: FIVE COPIES, ONE W





Vanishing Gradient

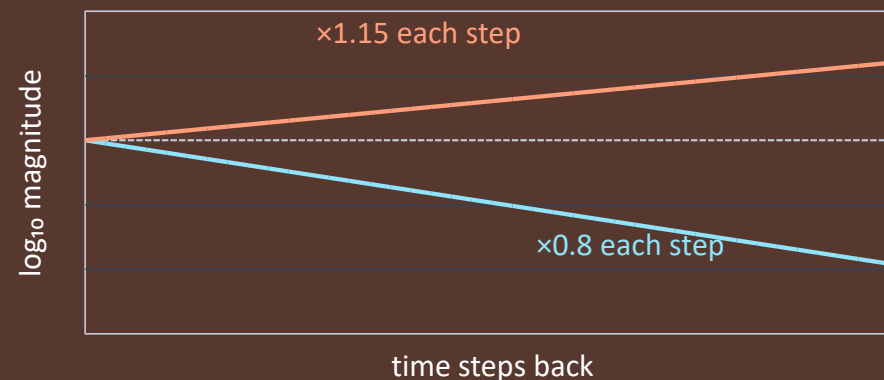
THE SAME MECHANISM AS DEPTH, ON THE TIME AXIS

- **the gradient through time** — a long product, one factor per step
- **twenty-four steps** — a twenty-four-layer network
- **worse than depth** — one matrix repeated, not different ones
- **below one it vanishes by ten steps** — above one it explodes
- **exploding has a fix** — clipping; vanishing has none

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_T} \prod_{t=2}^T \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}}$$

$\mathcal{L}$ the loss, $\mathbf{h}_1$ the first state, T the sequence length, the product is what vanishes

GRADIENT MAGNITUDE, BACK IN TIME



GRADIENT CLIPPING

If the gradient norm exceeds a threshold, rescale it. Three lines, and it cures the exploding case. Vanishing loses the information itself.

Cell State

A SECOND STATE, UPDATED BY ADDITION

- a **second state** — the cell state c , alongside the hidden state h
- its **update is addition** — not a matrix multiplication
- **left alone, c passes unchanged** — its gradient is multiplied by one
- a **residual connection in time** — a path with no weights on it
- the **rest of the LSTM** — controls what goes on and comes off

THE CONVEYOR BELT



Nothing is drawn on the belt yet. Left alone, the cell state passes through unchanged, and so does the gradient travelling back along it.

THE RESIDUAL CONNECTION, IN TIME

A residual connection is a path through depth on which the gradient is multiplied by one. The cell state is the same path through time, and it came first, in 1997.

Gates

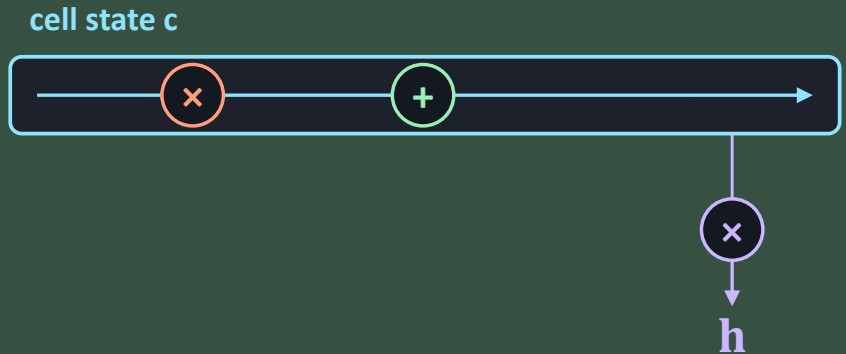
A SIGMOID AND AN ELEMENTWISE MULTIPLICATION

- a **gate** — a vector in $(0, 1)$, from a sigmoid layer on $[x, h]$
- **forget gate f** — multiplies the belt: how much to keep
- **input gate i** — how much of a tanh candidate g to add
- **output gate o** — how much of the cell state to expose as h
- **memory and output separate** — c can hold a value and say it once

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{x}_t, \mathbf{h}_{t-1}] + \mathbf{b}_f)$$

$\mathbf{f}_t$ the forget gate, σ squashes to $(0, 1)$, $[\mathbf{x}_t, \mathbf{h}_{t-1}]$ input and state stacked

THE FULL UNIT



all four from the same $[x, h]$ pair, by one Linear layer of width $4H$

Initialise the forget gate's bias to one: the unit then remembers by default and learns when to forget.

LSTM Equations

SIX LINES, AND THE GRADIENT ALONG THE BELT

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{x}_t, \mathbf{h}_{t-1}] + \mathbf{b}_f)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{x}_t, \mathbf{h}_{t-1}] + \mathbf{b}_i)$$

$$\mathbf{g}_t = \tanh(\mathbf{W}_g[\mathbf{x}_t, \mathbf{h}_{t-1}] + \mathbf{b}_g)$$

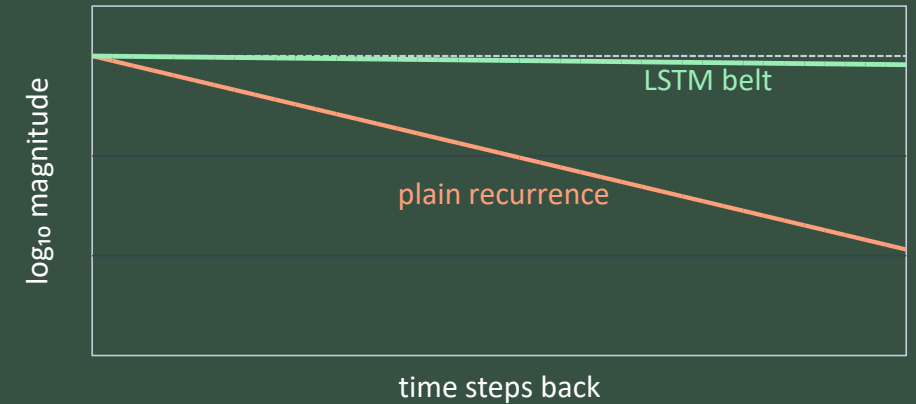
$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{x}_t, \mathbf{h}_{t-1}] + \mathbf{b}_o)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

As a layer: input width C , hidden width H , $4H(C + H + 1)$ parameters. The first four lines are one matrix product.

GRADIENT BACK IN TIME, GATE AT 0.99



THE DERIVATIVE ALONG THE BELT

Its leading term is the forget gate, elementwise, not a Jacobian to a power. The reach grows by orders of magnitude; it is not infinite.

Sizing an LSTM

WIDTH, DEPTH, DIRECTION, AND THE GRU

- **hidden width first** — sixteen to one hundred and twenty-eight
- **then depth** — two stacked layers is usually the most that helps
- **bidirectional only offline** — the backward pass reads the future
- **the GRU** — two gates instead of three, often as good and quicker

DECIDE · 2 MIN In the measured table on the right, the LSTM matches the perceptron's error at three times the parameters. For a substation controller, which of the three would you deploy, and why?

SENSIBLE DEFAULTS

hidden width	32 — grow only if underfitting
layers	1, or 2 at the most
clip the gradient	always, at norm 1
forget bias	initialise to 1

MEASURED IN EX_05 NOTEBOOK 04

model	parameters	held-out MSE
plain recurrent	305	0.00127
LSTM	1,169	0.00079
perceptron	417	0.00079

Self-Attention

EVERY POSITION READS EVERY OTHER, IN ONE STEP

- **do not carry it** — every position gets a direct path to every other
- **three vectors per position** — query, key, value, each learned
- **score query against key** — dot product, scale by $\sqrt{d}$, softmax
- **the output** — a weighted average of the values
- **all positions at once** — one parallel operation, no recurrence

$$\mathbf{Q} = \mathbf{XW}_q \quad \mathbf{K} = \mathbf{XW}_k \quad \mathbf{V} = \mathbf{XW}_v$$

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{QK}^T}{\sqrt{d}}\right) \mathbf{V}$$

$\mathbf{Q}, \mathbf{K}, \mathbf{V}$ queries, keys and values, d the width of one head, the weights sum to 1

WHAT RECURRENCE COSTS

path length	$T - 1$ steps from step 1 to step T
sequential	step t waits for step $t - 1$
bottleneck	one hidden vector holds the whole past

WHAT ATTENTION COSTS

path length	one, between any two positions
parallel	every position computed at once
the bill	quadratic in the sequence length

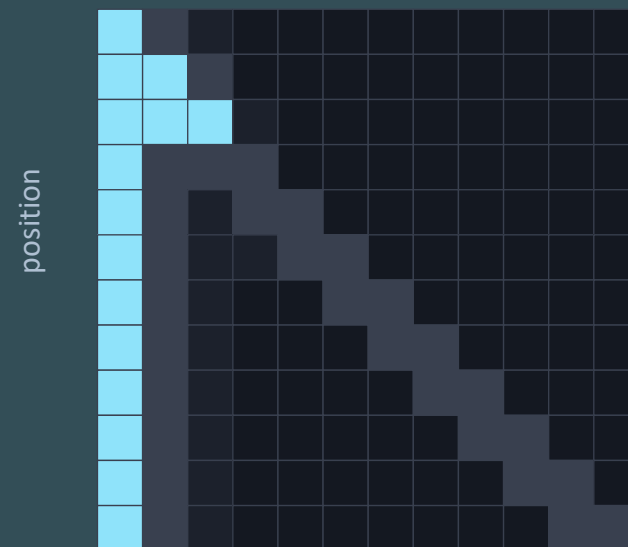
Attention Matrix

WHAT THE SOFTMAX PRODUCES

- a **square matrix** — one row per position, each row summing to one
- **readable** — which a dense layer is not
- **in notebook 04** — the same hour yesterday, and the last few hours
- **nobody told it to** — the daily cycle is the strongest signal
- a **weight says which values were used** — not why, not a cause

READ IT · 2 MIN In the matrix on the right, one row is brighter than the others at its left-hand end. Which hours is that position reading, and what in the data makes them worth reading?

MEAN ATTENTION, HELD-OUT WINDOWS



attends to position →

Bright is heavy. The bright left-hand columns are the same hour yesterday; the bright diagonal is the last few hours.

Heads and Position

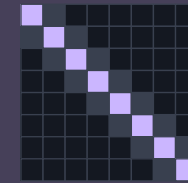
SEVERAL VIEWS, AND THE ORDER OF THE STEPS

- **one attention operation** — one notion of relevance
- **several heads** — several views in parallel, a slice of width each
- **attention is permutation equivariant** — order is invisible to it
- **on a graph that was the point** — on a time series it is a defect
- **positional encoding** — a vector per position, added to the input

$$\text{MultiHead}(\mathbf{X}) = [\mathbf{z}_1, \dots, \mathbf{z}_H] \mathbf{W}_o$$

head_i one attention, its own $\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v$, $[\cdot]$ concatenation, $\mathbf{W}_o$ mixes the heads

THREE HEADS, THREE VIEWS



recent hours



yesterday



a fixed lag

Different heads, different patterns of relevance, all computed at once.

THE SAME PROPERTY, OPPOSITE SIGN

Permutation equivariance made the graph network right for a power network. On a time series it is wrong, and the positional encoding is the patch.

Transformer Layer

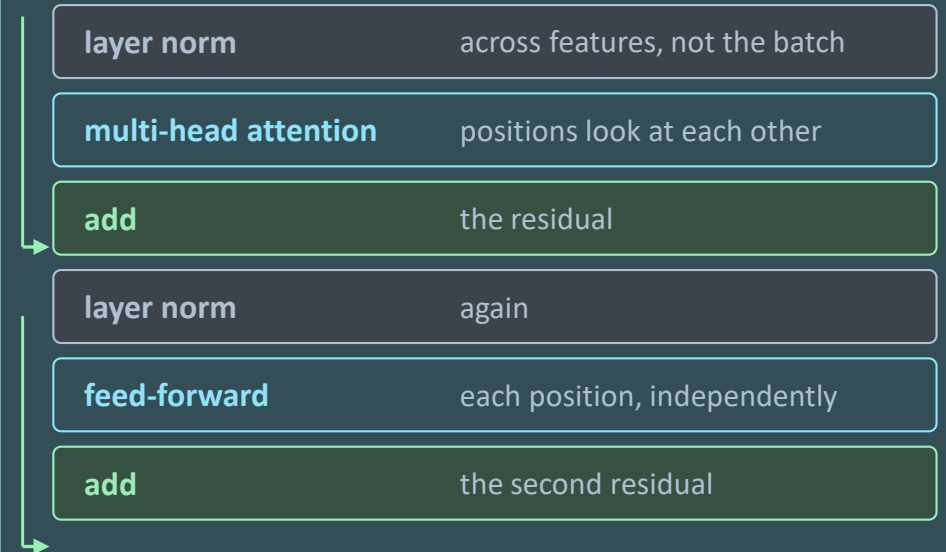
ATTENTION, A NETWORK PER POSITION, TWO RESIDUALS

- **multi-head attention** — the only place positions talk to each other
- **a feed-forward network** — applied to each position on its own
- residual connections and layer norm around both
- **stack it** — six layers in the original, ninety-six in a large one
- **nothing else changes with scale** — only the count

$$\mathbf{X} \leftarrow \mathbf{X} + \text{MHA}(\text{LN}(\mathbf{X})) \quad \mathbf{X} \leftarrow \mathbf{X} + \text{MLP}(\text{LN}(\mathbf{X}))$$

MHA multi-head attention, FFN a two-layer network per position, LN layer normalisation

ONE LAYER, TOP TO BOTTOM



Everything but the attention treats each position on its own, which is why the whole layer parallelises.

Transformer Models

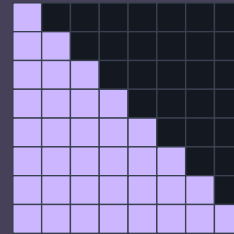
ENCODER — BERT



two positions hidden, predicted from the rest

Every position sees every other. Pre-trained by masked reconstruction, then a small head on your task.

DECODER — GPT



row i attends to columns $\leq i$

A position cannot see its own future. Every position is a training example.

VISION TRANSFORMER



one position per patch

Patches, flattened, are a sequence. Needs millions of images.

- **encoder** — reads the whole sequence, then answers about it
- **decoder** — predicts the next element — generation is that, repeated
- **the base model is not the assistant** — instruction tuning comes after

WHAT THIS SLIDE IS FOR

Enough to read a paper about one of these. Not enough to build one.

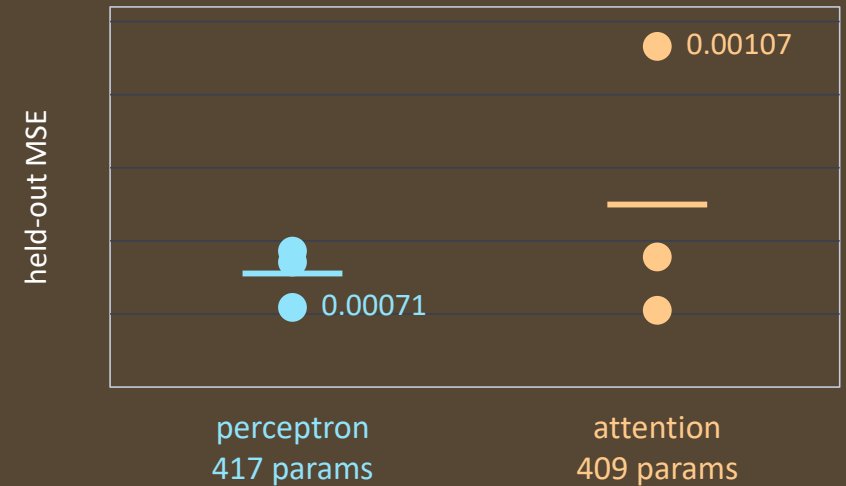
Seed Spread

SMALLER THAN THE SEED SPREAD IS NOT A RESULT

- **seed zero** — attention nearly twenty per cent ahead
- **retrain from three seeds** — and plot the spread
- **the bands overlap** — the margin belonged to the seed
- **stated carefully** — not worse; the evidence did not support it
- **three seeds cost four minutes** — one run each is not a comparison

TALK · 2 MIN Look at the two columns of points on the right. From seed zero alone, which model would you have reported as better, and what do the other two seeds do to that claim?

THREE SEEDS EACH



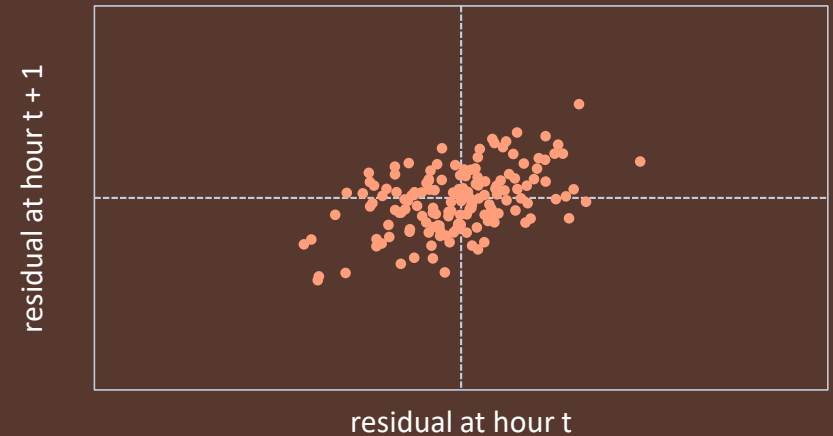
The distributions overlap. Seed zero of attention was the best run of the six; seed two was the worst.

Residual Check

A LOW ERROR THAT HIDES A FAILURE

- **the best model** — four times better than persistence, all four agree
- **plot each residual against the next** — the scatter leans upwards
- **lag-one autocorrelation 0.5** — half the next error is predictable
- **predictable, and not predicted** — the noise is AR(1) at 0.75
- **nothing else showed this** — not the MSE, not the loss curves
- **the failure stays in** — a check that always passes teaches nothing

RESIDUAL AT t AGAINST RESIDUAL AT $t + 1$



lag-1 autocorrelation: 0.50

If the model had extracted everything a twenty-four-hour window contains, this would be a formless cloud. It leans, and the lean is accuracy nobody collected.

Challenges

SHUFFLING BEFORE SPLITTING

overlapping windows share 23 of 24 values · split in time order · the held-out number is otherwise fiction

NO POSITIONAL ENCODING

attention cannot tell first from last · it still trains and reports a loss · add the encoding (Vaswani et al. 2017)

BIDIRECTIONAL ON A LIVE SIGNAL

the backward pass reads the future · offline analysis only · never on a controller

DATA AND ENERGY EFFICIENCY

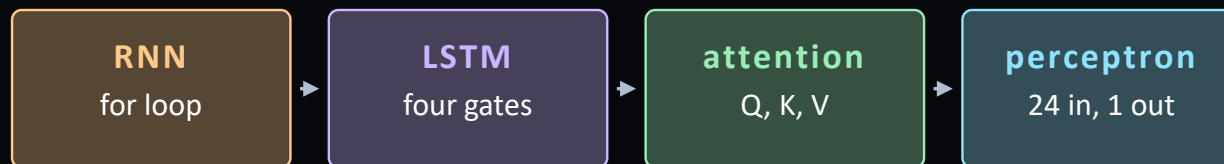
split	in time order
compare	against persistence first
plot	the residual autocorrelation
rerun	from three seeds
vary	the window length
test	on a different site or machine

The first four cost under ten minutes together and catch the leaked split, the meaningless baseline, the structured residual and the imaginary architecture win.

Exercise 5

NOTEBOOK 04 — FOUR MODELS, ONE PROBLEM

- **the data** — forty days of substation demand, generated on your machine: daily and weekly cycles, correlated noise
- **four models by hand** — the recurrence, the LSTM cell, single-head self-attention, a perceptron; five lines each
- **two baselines first** — persistence and the training mean; persistence is seven times better than the mean
- **then the two checks** — three seeds each, and the residual autocorrelation



THE QUESTIONS NOTEBOOK 04 ANSWERS

- § 5 does the LSTM repair the plain recurrence's error, and at what parameter cost?
- § 7 does attention's lead survive three seeds?
- § 9 what does the residual autocorrelation say that the MSE did not?

REPORT QUESTION

Four models within a factor of two of each other. Which would you deploy on a substation controller, and which single number decided it?

Key Takeaways

1 Weight sharing, a third time

One kernel per position, one rule per node, one cell per time step. The parameter count is free of the sequence length. Notebook 04, the recurrence.

2 Recurrence loses the gradient by construction

The gradient through time is a power of one matrix: it vanishes or explodes. Notebook 04, the plain recurrent model.

3 The LSTM adds an additive path

A cell state updated by addition, three gates controlling it. The derivative along the belt is the forget gate. Notebook 04, section 5.

4 Attention removes the carrying

Every position reads every other in one parallel step. The bill is quadratic cost and a positional encoding. Notebook 04, the attention model.

5 Measure before choosing

Three seeds dissolve attention's seed-zero win; the residual check finds structure a good MSE hid. Notebook 04, sections 7 and 9.

Next — L6.1: loss functions and gradients.

Questions

TEN QUESTIONS FOR THE ORAL EXAMINATION

- 1 Why does neither a convolution nor a dense layer fit a sequence problem?
UDL ch. 12.1
- 2 What does the hidden state of a recurrent network hold, and what is unrolling? *GBC ch. 10.1 – 10.2*
- 3 Why does the gradient vanish over time, and why is it worse than over depth? *GBC ch. 10.7*
- 4 What does the cell state change in an LSTM, and why is addition the key?
GBC ch. 10.10; Hochreiter & Schmidhuber (1997)
- 5 What do the forget, input and output gates each control, and why initialise the forget bias to one? *GBC ch. 10.10; Ex_05 notebook 04*
- 6 What are queries, keys and values, and which parts of self-attention are learned? *UDL ch. 12.2*
- 7 What does an attention weight tell you, and what does it not? *UDL ch. 12.2; Ex_05 notebook 04*
- 8 Why is a positional encoding not optional on a time series, and what does that say about permutation equivariance? *UDL ch. 12.3*
- 9 What are the parts of a transformer layer, and why does the layer parallelise? *UDL ch. 12.4; Vaswani et al. (2017)*
- 10 A model wins by ten per cent on one run. What do you check first, and what does the residual autocorrelation add? *Ex_05 notebook 04, sections 7 and 9*

NEXT

L6.1 — Loss Functions and Gradients

Every architecture this course uses is now in hand: dense, convolutional, graph, recurrent and attention. Next: where the loss comes from, and how the parameters move.

BEFORE THE NEXT SESSION

read	UDL ch. 5.1 – 5.3
run	Ex_05 notebooks 04 and 05
bring	your three-seed spread
think about	which model to deploy, on what number

Bring the seed spread. Two measured cases now where the fashionable architecture did not win, and that is worth more than either architecture.